2025 年 10 月 25 日
E. Togashi

強化学習を使った空調設備システムの最適制御

目次

1. はじめに 2
1.1 本資料の概要と目的 2
1.2 強化学習の枠組み 2
1.3 強化学習のプログラム開発のための環境構築 3
2. 迷路の脱出（強化学習の基礎） 4
2.1 問題の設定 4
2.2 プログラムの作成 4
1) 計算環境の構築 4
2) MazeEnvironment の定義 5
3) Agent の訓練 10
2.3 観察と報酬の設計法 12
1) マルコフ決定過程 12
2) Reward Hacking 13
3) Reward Shaping 13
4) 報酬設計の原則 13
3. 空調設備システムの最適化 1（強化学習の応用） 15
3.1 Environment の設計 15
1) 空調設備システムの全体像 15
2) 定式化 15
3) プログラムの実装 17
3.2 Agent の訓練プログラムの開発 22
3.3 学習の実行 24
1) 基本の学習 24
2) 報酬関数の変更 27
4. 空調設備システムの最適化 2（実システムへの展開に向けて） 29
4.1 エミュレータと強化学習キットの内容 29
1) 強化学習キットの導入 29
2) 環境クラス（Shizuku2EnvironmentSample） 30
3) 強化学習実行プログラム 35
4.2 Colab 環境での実行方法 37

1. はじめに

1.1 本資料の概要と目的

本資料は、強化学習を使って空調設備システムを最適化する手法について学ぶことを目的とする。本章では強化学習の基本的な枠組みと実践のための環境構築について説明する。第 2 章では、強化学習を使って簡単な迷路を解くことで、強化学習の基礎を理解する。第 3 章以降では、空調設備システムの最適化という具体的な問題について強化学習を応用する事例を示す。第 3 章では、ごく簡単な空調設備システムを題材とすることで設備側と強化学習の両方をプログラムし、全体の枠組みを学ぶ。第 4 章では、現実の建物への応用を見据え、BACnet で通信できる精緻な空調エミュレータシステムを対象に、ネットワーク越しに強化学習を適用する方法を学ぶ。

1.2 強化学習の枠組み

強化学習（RL: Reinforcement Learning）は機械学習の 1 手法である。環境（以下、*Environment*）と、その *Environment* の中で主体的に行動するエージェント（以下、*Agent*）をコンピュータの中に仮想的に作り、*Environment* と *Agenet* が相互に働きあう様子を再現することで、優れた *Agent* を鍛え上げる。

図 1.1 に強化学習の一般的な枠組みを示す。*Environment* の内部には状態（以下、*State*）があり、*Agent* はこの一部または全てを観察（以下、*Observation*）できる。*Agent* は *Observation* にもとづいて、*Environment* に対して行動（以下、*Action*）する。*Environment* は *Action* に応じて、その内部の *State* を変化させる。*Agent* は *Environment* から *Observation* の他に報酬（以下、*Reward*）を受け取る。この *Reward* の値が大きくなるように、*Agent* は *Action* を変えていく。

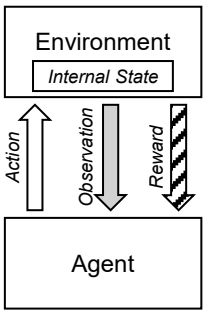


図 1.1　強化学習の一般的な枠組み

Agent が *Environment* に対して *Action* を与え、その *Action* を反映して更新された *Observation* と *Reward* を受け取り、次の *Action* を検討する。強化学習ではこの 1 サイクルをステップ（以下、*step*）と呼ぶ。また *Environment* と *Agent* の相互作用が終わるまでの一連の *step* の集合をエピソード（以下、*episode*）と呼ぶ。例えば将棋というゲームであれば、1 手 1 手の指し手が 1 つの *step* にあたり、勝敗が決するまでのすべての手の集合が 1 つの *episode* となる。なお、無限に *step* が連続し、*episode* を持たないような問題もある。

強化学習の枠組みを現実の問題に対してどのように当てはめるのかは設計の問題で、方法は無数にある。例えば、空調設備の制御の最適化を目的とする場合には、以下のような実装方法があるだろう。

建物の熱環境システムを *Environment* と設定し、*State* として外気条件、室内の温湿度、執務者の快適

性、エネルギー消費量、などを持つ。我々は省エネで快適な制御を導き出したいため、*Environment* は執務者が快適を感じ、エネルギーが少ない場合に、多くの *Reward* を出力するように設計する。この場合、*Agent* は優れた空調制御を模索する主体である。*Observation* として、*Environment* から外気条件、室内温湿度、快適性、エネルギー消費、などの情報と、その時の *Reward* を受け取る。そしてこれらの情報をもとに、*Reward* が最大化するように空調の発停や給気温度設定値などを決め、これを *Action* として *Environment* に伝える。

上記の枠組みの中で、*Agent* が *Reward* を最大化することができれば、省エネで快適な空調制御方法を獲得することができたということになろう。

1.3 強化学習のプログラム開発のための環境構築

すべてのプログラムを 1 から自分で書き上げることは困難のため、専門家たちが開発し、検証を重ねたライブラリを使うことが多い。機械学習に関わるライブラリは python で書かれたものが豊富にあるため、本例でも python を使ってプログラムを書く。

図 1.1 に示したように、強化学習は *Environment* と *Agent* が相互に働きかけることで学習を進める。*Environment* と *Agent* のそれぞれについて、汎用のライブラリがある。本例では、*Environment* を作るためのライブラリとして「Gymnasium (v.1.0.0)」を使い、*Agent* を鍛えるためのライブラリとして「Stable-Baselines3 (v.2.4.1)」を使う。

上記は個人所有のローカル PC でも動作する。しかし、OS の種別や PC の処理能力の違いなど、環境依存性によるトラブルを防止するため、本資料では Google Colaboratory（以下、Colab）を使うことを前提とする。高性能な GPU を使う場合などには有償のプランを契約する必要があるが、ある程度の性能で問題がなければ、google アカウントを持っているユーザーは誰でも無償で使うことができる。使い方の詳細は本資料では解説しない。

2. 迷路の脱出（強化学習の基礎）

2.1 問題の設定

図 2.1 に強化学習で解く迷路の構造を示す。

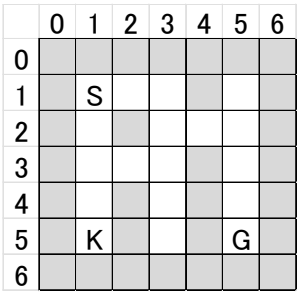


図 2.1　迷路の構造

行方向、列方向ともに 7 マス（0~6）ある。以降、それぞれのマスの座標を(行, 列)で表す。

白色が通路、灰色が壁を表す。*Agent* は通路のみを通ることができ、壁は通れない。*Agent* は最初、スタート位置を意味する「S」と書かれたマス（1, 1）にいる。ここからゴールを意味する「G」と書かれたマス（5, 5）へと最短の行動回数で移動することが *Agent* の目的である。ただし、ゴールに到達する前に鍵を意味する「K」と書かれたマス（5, 1）を通り、鍵を拾わなければならない。

Environment の *State* は「*Agent* が滞在しているマスの位置」と「*Agent* が鍵を入手済か否か」という 2 つの情報である。一般に *State* のすべてが *Agent* によって観察可能とは限らない。しかし、本例においてはこれらの情報を *Agent* も観察できると仮定する。従って *Observation* もこれらの 2 つの情報となる。

Agent はそれぞれの *step* で隣接するマスへの移動を試みることができるとする。従って、*Agent* の *Action* は上下左右のいずれか、4 通りである。

Agent は鍵を入手したときと、鍵をもってゴールに辿り着いたときに正の報酬を与えられる。また、これらの他に、各 *step* で行動をとるたびに負の報酬を与えられる。この負の報酬は *Agent* に対して、速くゴールへと移動することのインセンティブを与える。

2.2 プログラムの作成

Colab は 1 つの「ノートブック」という単位でプログラムコードを管理する。また、全体を適当なまとまりを持ったコードごとに区分して個別に実行することができる。この区分されたまとまりを「セル」と呼ぶ。

1) 計算環境の構築

まず、新しいノートブックを作り「+コード」をクリックしてセルを追加する。このセルに以下を入力して実行すれば gymnasium と stable-baselines3 がインストールされる（図 2.2）。この処理はセッションを起動するたびに 1 回だけ実行すれば良い。

```
!pip install "gymnasium==1.0.0" "stable-baselines3==2.4.1"
```



図 2.2　gymnasium と stable-baselines3 のインストール

2) MazeEnvironment の定義

強化学習を実行するために 2 つの大きなセルを用意する。一つはオリジナルの *Environment* を定義するセルで、もう一つはその *Environment* を使って *Agent* を訓練するセルである。*Environment* を定義するセルに入力するプログラムを以下に示す。

プログラムコード 2.1

```
import numpy as np
from gymnasium import Env, spaces

class MazeEnvironment(Env):
  """ 迷路の脱出を訓練するための環境クラス """

  # region 定数定義

  # スタート座標
  START: tuple[int, int] = (1, 1)

  # ゴール座標
  GOAL: tuple[int, int] = (5, 5)

  # 鍵の座標
  KEY: tuple[int, int] = (5, 1)

  # マップ：True が通路、False が壁
  MAP = np.array([
    [False, False, False, False, False, False, False ],
    [False,  True,  True,  True, False,  True, False ],
    [False,  True, False,  True,  True,  True, False ],
    [False,  True,  True,  True, False,  True, False ],
    [False,  True, False,  True, False,  True, False ],
    [False,  True, False,  True, False,  True, False ],
    [False, False, False, False, False, False, False ],
  ], dtype=bool)

  # 得点
  ACTION_PENALTY = -5   # 行動によるペナルティ
  GOAL_POINT = 600      # ゴール到達によるポイント
  KEY_BONUS = 50        # 鍵の取得によるボーナス

  MAX_STEPS = 50 # 最大ステップ数

  # endregion


  # 初期化処理=================================================
  def __init__(self):
    super().__init__()

```

```
43      # 状態をリセット
44      self._reset_state()
45
46      # 行動できる空間を定義（上：0、右：1、下：2、左：3）
47      self.action_space = spaces.Discrete(4)
48
49      # 観察できる状態空間の定義（現在の xy 座標,鍵入手状況）
50      self.observation_space = spaces.MultiDiscrete([self.MAP.shape[0], self.MAP.shape[1], 2])
51
52
53    # 状態の初期化処理*****************************************
54    def _reset_state(self):
55      self._current_step = 0              # ステップ数をリセット
56      self._current_point = self.START    # エージェントをスタート地点に戻す
57      self._has_key = False               # 鍵の所持状態をリセット
58
59
60    # 初期状態にリセットする（gymnasium 実装）*********************
61    def reset(self, seed=None, options=None):
62      """
63      環境を初期状態にリセットする。
64      Parameters
65      ----------
66      seed : int, optional
67          乱数の種（使わない場合は None）
68      options : dict, optional
69          追加のオプション（使わない場合は None）
70      Returns
71      -------
72      observation : ndarray
73          初期状態の観測
74      info : dict
75          追加情報（本例では使わない）
76      """
77      # 状態をリセット
78      self._reset_state()
79
80      # 初期位置での観測値（XY 座標,鍵の入手状況）を出力
81      return np.array([self._current_point[0], self._current_point[1], int(self._has_key)]), {}
82
83
84    # 環境を 1 ステップ進める（gymnasium 実装）*********************
85    def step(self, action):
86      """
87      環境を 1 ステップ進める。
88      Parameters:
89      ----------
90      action : 0,1,2,3 の値を取る
91           （上：0、右：1、下：2、左：3）
92      Returns:
93      -------
94      obs : ndarray
95          観測（現在の XY 座標,鍵の入手状況）
96      reward : float
97          報酬
98      terminated : bool
99          終了条件を満たしたか
100     truncated : bool
101         中断されたか（使わない場合は False）
102     info : dict
103         追加情報（本例では使わない）
104     """
105
106     # ステップ数を増やす
107     self._current_step += 1
108
109     # 移動を検討**********************************
110     row, col = self._current_point
111
```

```
112     # 次の座標候補
113     if action == 0: row -= 1   # 上
114     elif action == 1: col += 1 # 右
115     elif action == 2: row += 1 # 下
116     else: col -= 1             # 左
117
118     # 次の座標候補が「壁」でなければ移動
119     if self.MAP[row, col]:
120       self._current_point = (row, col)
121
122     # 報酬の計算**********************************
123     reward = 0
124
125     # 鍵の取得を判定
126     if not(self._has_key) and self._current_point == self.KEY:
127       self._has_key = True
128       reward += self.KEY_BONUS
129
130     # 行動ペナルティを加算
131     reward += self.ACTION_PENALTY
132
133     # ゴール到達の報酬を加算
134     terminated = (self._current_point == self.GOAL and self._has_key)
135     if terminated:
136       reward += self.GOAL_POINT
137
138     # 観察（XY 座標,鍵の入手状況）*****************
139     obs = np.array([self._current_point[0], self._current_point[1], int(self._has_key)])
140
141     # 報酬と観察結果を出力
142     truncated = (self._current_step >= self.MAX_STEPS) # 最大ステップを超えたか否か
143     return obs, reward, terminated, truncated, {}
144
145
146   # 環境の状態を描画する（gymnasium 実装）**********************
147   def render(self):
148     """
149     環境の状態を描画する。
150     """
151     # すべてのマスを壁'#'で埋める
152     render_map = np.full(self.MAP.shape, '#')
153
154     # 壁でない部分を、通路' 'で上書きする
155     render_map[self.MAP] = ' '
156
157     # キーの取得前ならばキーの位置を'K'で上書きする
158     if not self._has_key:
159       render_map[self.KEY] = 'K'
160
161     # ゴール地点を'G'で上書きする
162     render_map[self.GOAL] = 'G'
163
164     # 最後に、エージェントの現在地を'A'で上書きする
165     render_map[self._current_point] = 'A'
166
167     # 1 行ずつ文字列に変換して表示
168     for row in render_map:
169       print(''.join(row))
170
171     # 見やすいように区切り線を入れる
172     print("-" * self.MAP.shape[1])
```

プログラムの説明をする前に、この *Environment* がどのように応答するのかを確認しておく。新しいセルを作成し、以下のプログラムを入力して実行する。

プログラムコード 2.2

```
# 環境を作成・初期化して状態を描画
env = MazeEnvironment()
obs, info = env.reset()
env.render()

# 報酬
total_reward = 0
print("Action - 0:上, 1:右, 2:下, 3:左 (q で終了)")

# メインループ
while True:
  # ユーザーからの入力を受け取る
  user_input = input("Action: ")

  # 'q'が入力されたらテストを終了
  if user_input.lower() == 'q':
    print("終了します。")
    break

  # 入力が正しいかチェック
  try:
    action = int(user_input)
    if action not in [0, 1, 2, 3]:
      print("無効なアクション。0, 1, 2, 3 のいずれかを入力")
      continue
  except ValueError:
    print("無効な入力。数値を入力")
    continue

  # 環境を 1 ステップ進める
  obs, reward, terminated, truncated, info = env.step(action)
  total_reward += reward

  # 更新された状態を描画
  env.render()

  # 現在の合計報酬を表示
  print(f"即時報酬: {reward}  合計報酬: {total_reward}")

  # 終了条件をチェック
  if terminated:
    if env._current_point == env.GOAL:
      print("ゴール到達により終了")
    break
```

セルを実行すると図 2.3 のような応答が得られる。迷路の構造がテキスト表示されており、*Agent* の位置は A で表示される。*Action* として上下左右のいずれかを決める必要があり、それぞれ 0, 1, 2, 3 の数値が対応している。例えば「下」を意味する「2」を入力すると *Agent* が 1 マス下に移動する。数字の入力を続けることで *Agent* が移動していくことを確認されたい。

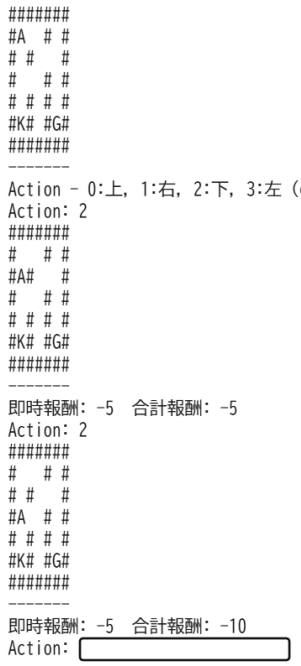


図 2.3　迷路 Environment の応答

Reward として「即時報酬」と「合計報酬」の 2 つが表示される。即時報酬（immediate reward）はそれぞれの *step* で与えられる報酬を意味する。合計報酬（total reward）は全ての *step* の報酬を合算した値である。一般に強化学習では単に合計報酬を最大化せず、将来の *Reward* の価値を割り引いて評価する。つまり、*n* 番目の *step* の *reward* を r_n としたとき、次式で表される割引累積報酬 V を最大化するように学習が進む。

$$V = \sum \gamma^n r_n \qquad 2.1)$$

ここで γ [-]は割引率で 0.0~1.0 の値を取る。割引率を小さくすれば将来時点の不確実性の高い *Reward* の価値を小さく見積もることになる。逆に 1.0 とすれば、現在の *Reward* と将来の *Reward* の価値を全く同等とみなすことになり、合計報酬が割引累積報酬に一致する。

以上の応答を理解した上でプログラムコード 2.1 を解説する。gymnasium で新たな *Environment* を定義するためには *Env* クラスを継承した派生クラスを作成する（4 行）。本例では *MazeEnvironment* というクラスを定義している。

Env 派生クラスでは *step*、*reset*、*render* という 3 つのメソッドを実装する。*step* には *Agent* からの *Action* を受け取り、*State* を更新し、新しい *Observation* と *Reward* を出力するための処理を記述する。*reset* には 1 つの *episode* が終わったときの処理を記述する。*render* には途中経過を可視化する処理を記述するが、これはユーザーの確認が目的であり、学習するために必須ではない。

定数宣言

プログラムの冒頭（7-36 行）で *Environment* に関わる定数を宣言する。迷路のマップは二次元の配列で表し、通路を True、壁を False とする。

初期化処理

39~50 行はクラスの初期化処理である。まず、53~57 行で定義したメソッドを呼ぶことで *Environment*

の *State*、つまり *step* 数、*Agent* の位置、鍵の有無を初期化する。

Environment に対してどのような *Action* が許されるのかを 47 行で定義する。spaces.Discrete は行動が離散値であることを意味する。本例では上下左右の 4 つの行動があるため、引数として 4 を与える。離散値の他にも連続量を設定することもでき、これについては後述の空調設備システムの最適化の例で具体例を示す。

Environment からどのような *Observation* を受け取れるのかを 50 行で定義する。spaces.MultiDiscrete は、複数の離散値が観察できることを意味する。本例では *Agent* がいる行方向と列方向の座標に加えて鍵の取得状況があり、3 つの離散値が観察できる。なお、鍵が入手済みのときは 1、未入手のときは 0 という数値を割り当てる。

reset メソッド

61~81 行は、新しい *Episode* を始めるときに呼び出す *reset* メソッドの定義である。*reset* メソッドは各種の状態値などを初期化すると同時に、最初の *Observation* の結果を出力する。*Agent* はこの結果を使うことで初回の *Action* を決定する。

reset メソッドの引数として乱数シード（*seed*）が与えられる。しかし、本例では *Environment* は確率的に振る舞わないので使わない。具体例は後述の空調設備システムの最適化で示す。

81 行で初期の *Observation* の結果を出力する。int()は鍵の入力状況の True, False を 1, 0 に変換するための関数である。

step メソッド

85~145 行は、*step* ごとに呼び出すメソッドの定義である。*step* メソッドは *Agent* から受け取った *Action* にもとづいて *State* を更新して *Reward* を計算する。さらに *Agent* に対してこの *Reward* と *Observation* を出力する。

本例では 50 ステップで強制的に *Episode* を終了させる。このため、現在の *step* 数を 107 行で増やして保持する。

Action の値に応じて上下左右のどちらに動こうとしているのかを判定し、もしも移動先が壁ではなく通路であれば、*Agent* の位置を更新する（109~120 行）。

鍵の取得（125~128 行）、行動による *step* ごとのペナルティ（131 行）、ゴール到達（134~136 行）、のそれぞれの *Reward* を合計し、現在の *Observation*（139 行）とともに 143 行で出力する。terminated と truncated はそれぞれ終了あるいは中断の判定であり、True の場合には次の *episode* へ移る。本例では鍵を持ってゴールにたどり着けば終了、最大 *step* 数に到達すると中断、となる。

146 行以降は状態表示処理である。

3) Agent の訓練

新しいセルを用意して以下のコードを入力する。

プログラムコード 2.3

```
from stable_baselines3 import PPO
from stable_baselines3.common.env_checker import check_env
from stable_baselines3.common.utils import set_random_seed

# 乱数の seed を固定
SEED = 1
set_random_seed(SEED)

```

```
 9 # 環境の準備*****************
10 env = MazeEnvironment()
11 # 自作した環境が Gymnasium のルールに従っているかチェック
12 check_env(env)
13
14 # モデルの定義と学習*********
15 # PPO というアルゴリズムで、シンプルな多層ニューラルネットワーク(MlpPolicy)を使う
16 model = PPO(
17   "MlpPolicy",
18   env,
19   verbose=1,
20   ent_coef=0.02, # エントロピー係数 (0 で確定的、0.001-0.02 程度が一般的)
21   seed=SEED , # seed を指定
22 )
23
24 # 環境を操作して学習*********
25 print("学習を開始...")
26 model.learn(total_timesteps=100000, progress_bar=True)
27 print("学習完了")
28
29 # 学習済みエージェントのテスト*********
30 print("¥n 学習済みエージェントの動きを確認...")
31
32 obs, info = env.reset()
33 env.render()
34
35 # 1 エピソード実行して、エージェントの動きを観察
36 terminated = False
37 truncated = False
38 ttl_rwd = 0
39 while not(terminated or truncated):
40   # モデルに現在の状態(obs)を渡し、最適な行動(action)を予測
41   action, _states = model.predict(obs, deterministic=True)
42
43   # 予測された行動を実行
44   obs, reward, terminated, truncated, info = env.step(int(action))
45   ttl_rwd += reward
46
47   # 画面を更新
48   env.render()
49   print("合計報酬:{} 報酬:{}".format(ttl_rwd,reward))
50
51 env.close()
```

Agent の学習は確率的に進められるため、常に同じ学習結果が得られるとは限らない。しかし、本資料は学習を目的とした教材であり、同じ結果が得られることが望ましいため、乱数のシードを強制的に固定する（6~7 行）。

10 行で先ほど定義した迷路クラスのインスタンスを作り、12 行でその構造を検証する。

16~22 行は学習につかうアルゴリズムの指定である。*Agent* を学習させる方法は 1 つだけではなく、stable_baselines3 は PPO、A2C、DQN など、多くの学習方法に対応している。本例では Proximal Policy Optimization（PPO）を使う。第 1 引数（*MlpPolicy*）で、多層ニューラルネットワーク（MLP: Multi Layer Perceptron）を使うことを指示する。第 2 引数は *Environment* の指定で、ここでは先に用意した *MazeEnvironment* のインスタンスを与える。第 3 引数の verbose は学習過程をどの程度、画面表示するかの設定で、学習結果には影響が無いのでとりあえず 1 としておけば良い。第 4 引数の ent_coef はエントロピー係数である。*Agent* は未知の行動を取ることで新しい領域を探索する。エントロピー係数はこの探索の積極性を左右する係数で、値が大きいほど未知の行動を取るようになる。

26 行で学習を始める。本例では学習回数は 100,000 回とした。

29 行以降は学習済の *Agent* の動きを確認するための可視化処理で、説明は省略する。

プログラムを実行すると図 2.4 に示すように学習の進捗が表示される。「ep_len_mean」と「ep_rew_mean」は 1 エピソードあたりの平均 *step* 数と平均 *Reward* である。学習を始めたばかりの状態では、*Agent* はマップ内をウロウロと歩き回り、ゴールにたどり着く前に 50 回の最大 *step* に到達してしまう。このため、平均 *step* 数は 50 に近い値となる。また平均 *Reward* も小さい。学習が進むとゴールできることが多くなり平均 *step* 数が 14 に近づく。

```
|    value_loss           | 5.29e+03    |
-----------------------------------------
-----------------------------------------
| rollout/                |             |
|    ep_len_mean          | 49.6        |
|    ep_rew_mean          | -162        |
| time/                   |             |
|    fps                  | 519         |
|    iterations           | 10          |
|    time_elapsed         | 39          |
|    total_timesteps      | 20480       |
| train/                  |             |
|    approx_kl            | 0.011937829 |
|    clip_fraction        | 0.076       |
|    clip_range           | 0.2         |
|    entropy_loss         | -1.25       |
|    explained_variance   | 0.000593    |
|    learning_rate        | 0.0003      |
|    loss                 | 1.51e+03    |
|    n_updates            | 90          |
|    policy_gradient_loss | -0.009      |
|    value_loss           | 3.68e+03    |
-----------------------------------------
-----------------------------------------
| rollout/                |             |
|    ep_len_mean          | 49          |
|    ep_rew_mean          | -136        |
| time/                   |             |
```

図 2.4　学習の進捗表示

学習済の *Agent* の動きを追えば、最短距離で鍵を取得してゴールにたどり着けていることが確認できる。このときの *step* 数は 14、報酬は 595 である。なお、100,000 回の学習の最終回でも平均 *step* 数は 14 よりも少し大きく、平均 *Reward* は 595 よりも少し低い値になる点は重要である。学習の過程では *Agent* は常に別の優れた解を探索し続けるため、現在、得られている最適解からは外れた行動を取るためである。学習結果の確認の場面では 41 行に示すように deterministic=True と設定することで、探索のための確率的な行動を禁止する。

2.3 観察と報酬の設計法

不適切な *Observation* や *Reward* とすると学習は成功しない。注意べきいくつかの要点を以下で解説する。

1) マルコフ決定過程

一般的に強化学習では、*Environment* はマルコフ決定過程（以下、MDP: Markov Decision Process）に従うことを仮定する。これは「次の *step* の状態は現在の状態と行動のみから決定される」ということを意味する。従って、*Agent* は 1 つの *Observation* から 1 つの最適な *Action* しか学習し得ない。

例えば本例では *Agent* の座標に加えて、鍵の入手状況を *Observation* とした。仮に鍵の入手状況が与えられなければ *Agent* の学習は成功しない。これは *Agent* が座標(4, 1)にいることを想定すれば容易に理解できる。この座標にいるとき、*Agent* は鍵の入手前であれば下方向に進み、鍵を取得すべきである。逆に

鍵が入手済みであれば上方向に進み、ゴールへ向かうべきである。つまり、鍵の入手状況に応じて最適な行動は変わる。もしも *Observation* として座標のみしか与えられなければ、座標(4, 1)に対して 2 つの最適な *Action* が存在することになるため、*Agent* の学習は失敗する。

従って、*Observation* の内容を検討する際には、MDP という観点において、*Agent* が判断をして行動を決定するために必要な十分な情報が与えられているのかどうかを検討しなければならない。特に空調設備システムの最適化という場面では、明らかに熱的な遅れの影響がある。単に瞬時の温度を与えるだけでは、*Agent* はこのような熱的遅れを踏まえた学習はできないだろう。

2) Reward Hacking

不適切な報酬設計により、*Agent* が本来の目的とは異なる行動を学習することを一般的に報酬ハッキング（Reward Hacking）と呼ぶ。この現象を確認するため、先に定義した *MazeEnvironment* の鍵の取得に関わるプログラムを以下のように変更されたい。

プログラムコード 2.4

```
    # 鍵の取得を判定
    #if not(self._has_key) and self._current_point == self.KEY:
    if self._current_point == self.KEY:
      self._has_key = True
      reward += self.KEY_BONUS
```

変更後に学習をすると *Agent* が鍵のマスにとどまり続ける様子が確認できる。鍵を取得することのボーナスを繰り返し取得できるようにしたため、ゴールへ向かうという大局的な目的を見失ったためである。

3) Reward Shaping

逆に、次のようにプログラムを変更して鍵取得のボーナスを 0 とすれば何が起きるだろうか。

プログラムコード 2.5

```
# 得点
ACTION_PENALTY = -5   # 行動によるペナルティ
GOAL_POINT = 600      # ゴール到達によるポイント
#KEY_BONUS = 50         # 鍵の取得によるボーナス
KEY_BONUS = 0        # 鍵の取得によるボーナス
```

この場合も学習は成功しない。鍵を拾ってゴールに辿り着くという一連の行動を完遂して初めて *Reward* が与えられる。しかし、50 回という限られた *step* 数では *Agent* はこのような複雑な行動に 1 度も成功しないまま、迷路内をさまよい続けるためである。このような問題を回避するためには、ゴール到達時にまとめて大きな *Reward* を与えるのではなく、*Action* の方向性が良いときに中間的に適度な *Reward* を与えることが推奨される。これは Reward Shaping と呼ばれるテクニックである。一般に Reward Shaping は学習の速度を上げる効果を持っている。鍵の取得によるボーナスは「鍵を取りゴールに向かう」という目的の中間到達点において *Reward* を与えることになるため、典型的な Reward Shaping と言える。

4) 報酬設計の原則

迷路の形状を見ると、最初に右方向に進むのではなく、下方向に進むほうが良い。では Reward Shaping として、第 1 *step* で下方向に向かった場合に *Reward* を与えることは適切だろうか。これは報酬設計の原則に反しており、推奨されない。この原則は以下のように表現される。

> *The reward signal is your way of communicating to the robot what you want it to achieve, not how you want it achieved*. (Sutton & Barto 2018[1])

つまり、*Reward* は *Agent* に対して何（What）を達成したいのかを伝えるべきであり、どのように（How）達成したいのかを伝えてはならない。確かに本例の場合には第 1 *step* で下方向に進むことが最適であることは自明である。しかし、人間が真の最適解を知り得ない問題を *Agent* に学習させるときに、人間がその限定的な知識をもとに優れていると判断した *Action* の方法（How）を *Agent* に強いてしまっては、*Agent* が人間の知恵の及ばない優れた *Action* を学習することを阻害してしまう。

3. 空調設備システムの最適化 1（強化学習の応用）

本章では、空調設備システムの最適化という、より具体的な場面に強化学習を応用する方法を学ぶ。

3.1 Environment の設計

1) 空調設備システムの全体像

我々が模擬しようとしている空調設備システムの全体像を図 3.1 に示す。

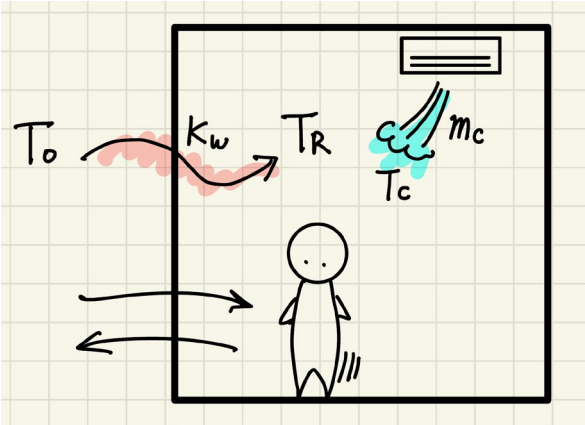


図 3.1　空調設備システムの全体像

夏季の冷房システムが計算対象である。室内温度 T_R [°C]は状態変数で、屋外からの熱流と空調機から給気される冷温風によって値を変える。屋外からの熱流は、外気温度 T_o [°C]と壁の熱抵抗（伝熱係数）K_w [kW/K]によって決まる。空調機から供給される熱は給気温度 T_c [°C]と給気風量 m_c [kg/s]によって決まる。設備システムは充電量 E_B [kWh]の電池を持っており、空調機の処理熱量に応じて充電量が減る。室内には執務者が滞在しているが、室内の快適性によっては部屋から退出する。

Agent は外気温度 T_o、室内温度 T_R、電池の充電量 E_B を *Observation* できる。これらの情報を頼りに、*Action* として空調機の給気温度 T_c と給気風量 m_c を決める。*Environment* は *Action* の内容に応じて室温 T_R を更新し、執務者の不満足率を計算する。執務者はこの不満足率にもとづいて確率的に部屋から退出する。また、*Environment* は、空調機の稼働状況から消費電力を計算し、充電量 E_B を減少させる。

1 つの *episode* は、以下の 3 つのいずれかの条件によって終了する。

1）室内環境が悪化して執務者が退室する

2）電池の充電量が 0 となり、空調が使えなくなる

3）10 時間が経過する

Reward は各 *step* における執務者の（1-不満足率）とする。この他、終了条件 3（10 時間の経過）で *episode* が終了した場合には、（0.3×残存充電量）をボーナスの *Reward* として受け取ることができる。

以上により、*Agent* は、外気条件に応じて、電力の消費を抑えつつ、執務者が部屋を去らない程度に室内環境の快適性を維持できる空調の制御方法を学習する。

2) 定式化

・室温変動

室温変動は時間 t [sec]に関して次の微分方程式で表すことができる。

$$C_R \frac{dT_R}{dt} = K_w\big(T_o - T_R(t)\big) + m_c c_{pa}\big(T_c - T_R(t)\big) \quad 3.1)$$

ここで C_R [kJ/K]は室の熱容量、c_{pa} [kJ/(kg·K)]は空気の定圧比熱で、それぞれ 1000 kJ/K、1.0 kJ/(kg·K)の一定値とする。

式 3.1 を解くと式 3.2~3.4 となる。

$$T_R(t) = \left(T_R(0) - \frac{b}{a}\right)\exp(-at) + \frac{b}{a} \quad 3.2)$$

$$a = \frac{K_w + m_c c_{pa}}{C_R} \quad 3.3)$$

$$b = \frac{K_w T_o + m_c c_{pa} T_c}{C_R} \quad 3.4)$$

外気温度はランダムに変化するとし、次式に示すように平均 30、標準偏差 2 の正規乱数とする。ただし、取りうる温度の範囲は 25~35℃ とする。

$$T_o = \max\big(25, \min\big(35, N(30,2)\big)\big) \quad 3.5)$$

・不満足率

不満足率は Fanger による予想温冷感申告（以下、PMV: Predicted Mean Vote）と予測不満足者率（PPD: Predicted Percentage of Dissatisfied）を応用して計算する。Fanger の PMV は 6 つの熱的要素にもとづいて計算されるが、本例のモデルは乾球温度のみが計算対象であるため、放射温度は乾球温度とみなし、残りの 4 要素は固定値を使う。具体的には、相対湿度は 50 %、相対気流速度は 0.2 m/s、着衣量は 0.8 Clo、活動量は 1.1 met とした。この前提にもとづき、乾球温度（=放射温度）のみの関数として PMV の近似式を求めると次式となる。

$$PMV_T = 0.2981 T_R - 7.1882 \quad 3.6)$$

この近似値を PPD の計算式に代入することで、PPD の近似値を次式で推定する。

$$PPD_T = 1.0 - 0.95\exp(-0.03353 PMV_T^4 - 0.2179 PMV_T^2) \quad 3.7)$$

PPD_T は 0.0 から 1.0 の値をとり、これを部屋の温度状態にもとづく執務者の不満足率とみなす。上式を使って乾球温度から不満足率を計算すると図 3.2 が得られる。不満足率は概ね 24℃ で最小化し、14℃ 以下ないしは 34℃ 以上になるとほぼ確実に不満となる。

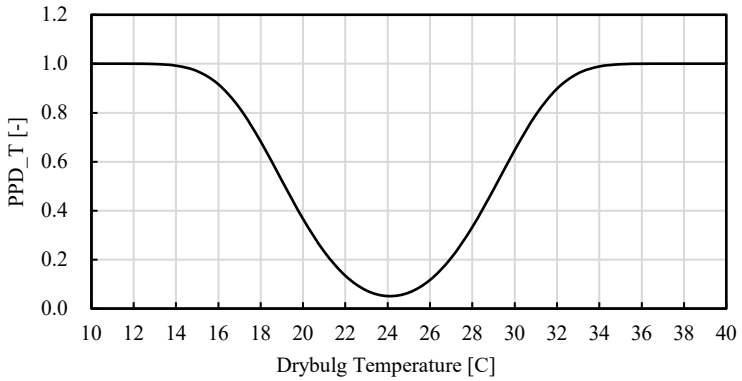


図 3.2　乾球温度と不満足率の関係

0.0~1.0 の値を取る一様乱数 $U(0,1)$と執務者の不満者率を使い、次式が成立するとき執務者を退室させる。

$$(PPD_T)^2 < U(0,1) \tag{3.8}$$

・消費電力

空調機の消費電力 E_c [kW]を次式で計算する。

$$E_c = \frac{m_c c_{pa} |T_c - T_R(t)|}{COP} \tag{3.9}$$

COP [-]は空調機の COP で、3.0 で一定とする。給気風量 m_c と給気温度 T_c は *Agent* の *Action* であり、それぞれ 0.2~1.0 kg/s、10~40 °C の値を選択できるとする。室温を 25 °C、給気温度を 10 °C、給気風量を 1 kg/s とすれば、消費電力 E_c は 5 kW となる。これを定格条件とし、電池の容量はこれの 4 時間相当、すなわち 20 kWh とする。*episode* の開始時点において電池は満充電（$E_{B,0}$=20 kWh）とし、*step* ごとに次式で更新する。

$$E_{B,n+1} = \max\left(0, E_{B,n} - E_c \frac{\Delta t}{3600}\right) \tag{3.10}$$

ここで Δt [sec]は計算のタイムステップで、3600 秒とする。

・報酬と価値

n 回目の *step* での Reward（r_n）は次式で表される。

$$r_n = \begin{cases} 0 & (no\ occupant\ or\ no\ battery) \\ (1 - PPD_{T,n}) & (0 < n < 9) \\ (1 - PPD_{T,n}) + 0.3E_{B,n} & (n = 9) \end{cases} \tag{3.11}$$

執務者が不在か、電池が切れた場合には *Reward* は 0 である。その他の場合には満足率（1-不満足率）を報酬とするが、最終タイムステップ（n=9）において電池が残っている場合には残存容量に 0.3 を乗じた値をボーナスとして加算する。割引率は 1.0 とする。空調の最適化という目的においては、この設定が適切と思われる。

3) プログラムの実装

前節で定式化したシステムをプログラムとして実装する。新規にノートブックを作成して新しいセルに以下のコードを入力する。

プログラムコード 3.1

```
"""
HVACEnvironment.py
空調の最適制御を訓練するための環境クラス
"""
import time
import numpy as np
import random
from gymnasium import Env, spaces

class HVACEnvironment(Env):
  """ 空調の最適制御を訓練するための環境クラス """

  # region 定数定義

  # --- シミュレーション設定 ---
  MAX_STEP = 10   # 最大ステップ数
  DELTA_T = 3600  # 時間間隔[sec]
```

```
18
19  # --- 物理モデル定数 ---
20  K_W = 1.0  # 壁の伝熱係数[kW/K]
21  CPA = 1.0  # 空気の定圧比熱[kJ/(kg・K)]
22  C_R = 1000 # 室の熱容量[kJ/K]
23  COP = 3.0  # 空調機の COP[-]
24
25  # --- 標準化用定数 ---
26  TEMP_MAX = 50.0  # 温度標準化用の最大値[C]
27  BATTERY_MAX = 5.0 * 4.0  # 最大充電量[kWh](定格 5kW の 4h 相当)
28
29  # endregion
30
31  # 初期化処理************************************************
32  def __init__(self):
33    super(HVACEnvironment, self).__init__()
34
35    # 状態初期化
36    self._reset_state()
37
38    # 観察できる状態空間の定義（外気温度,室内温度,残電力量を 0.0-1.0 の値に標準化）
39    self.observation_space = spaces.Box(
40      low=np.array([0.0, 0.0, 0.0], dtype=np.float32),
41      high=np.array([1.0, 1.0, 1.0], dtype=np.float32),
42      dtype=np.float32)
43
44    # 行動できる空間を定義（風量,給気温度を 0.0-1.0 の値に標準化）
45    self.action_space = spaces.Box(
46      low=np.array([0.0, 0.0], dtype=np.float32),
47      high=np.array([1.0, 1.0], dtype=np.float32),
48      dtype=np.float32)
49
50  # 状態の初期化処理******************************************
51  def _reset_state(self):
52    self._no_occupant = False # 執務者の在不在
53    self._current_step = 0 # ステップ
54    self._room_temp = 24.0 # 室温[C]
55    self._outdoor_temp = 35.0 # 外気温度[C]
56    self._battery = self.BATTERY_MAX # 残電力量[kWh]
57
58  # 初期状態にリセットする（gymnasium 実装）*********************
59  def reset(self, seed=None, options=None):
60    """
61    環境を初期状態にリセットする。
62    温度や電池残量などの状態を初期値に戻し、初期観察値を返す。
63    Parameters
64    ----------
65    seed : int, optional
66    乱数の種（使わない場合は None）
67    options : dict, optional
68    追加のオプション（使わない場合は None）
69    Returns
70    -------
71    observation : ndarray
72    初期状態の観察（外気温・室温・電池残量）を [0.0〜1.0] に標準化した配列
73    info : dict
74    追加情報（本例では使わない）
75    """
76    # 状態初期化
77    self._reset_state()
78
79    # 乱数シード初期化
80    if seed is None:
81      random.seed(int(time.time() * 1000))
82    else:
83      random.seed(seed)
84
85    # 観察を出力
86    return self._get_observation(), {}
```

```
87
88    # 観察結果を取得する**************************************
89    def _get_observation(self):
90      return np.array([
91        self._outdoor_temp / self.TEMP_MAX,
92        self._room_temp / self.TEMP_MAX,
93        self._battery / self.BATTERY_MAX
94      ], dtype=np.float32)
95
96    # 環境を１ステップ進める（gymnasium 実装）*********************
97    def step(self, action):
98      """
99      環境の１ステップを進める。
100     Parameters:
101     ----------
102     action : list or ndarray
103     [風量（0〜1), 給気温度（10〜40)]
104     Returns:
105     -------
106     obs : ndarray
107     観測（外気温、室温、電池残量）
108     reward : float
109     報酬
110     terminated : bool
111     終了条件を満たしたか
112     truncated : bool
113     中断されたか（使わない場合は False）
114     info : dict
115     追加情報（本例では使わない）
116     """
117     # ステップ追加
118     self._current_step += 1
119
120     # 行動の取得
121     air_flow = 0.2 + 0.8 * np.clip(action[0], 0, 1.0) # ファン風量[kg/s]
122     supply_temp = 10.0 + 30.0 * np.clip(action[1], 0.0, 1.0) # 給気温度[C]
123
124     # 消費電力[kWh]を計算
125     load = (abs(supply_temp - self._room_temp) * self.CPA * air_flow)
126     power = load / self.COP * (self.DELTA_T / 3600)
127     self._battery = max(0, self._battery - power)
128
129     # 室温を更新
130     a = (self.K_W + self.CPA * air_flow) / self.C_R
131     b = (self.K_W * self._outdoor_temp + self.CPA * air_flow * supply_temp) / self.C_R
132     self._room_temp = (self._room_temp - b/a) * np.exp(-a * self.DELTA_T) + b / a
133
134     # 外気温度を更新
135     self._outdoor_temp = random.gauss(mu=30, sigma=2)
136     self._outdoor_temp = max(25, min(35, self._outdoor_temp))
137
138     # 執務者の不満足度を計算
139     # 相対湿度 50%,気流速度 0.2m/s,着衣 0.8Clo,活動量 1.1met に固定したときの PPD 近似値
140     pmv = self._room_temp * 0.2981 - 7.1882
141     ppd = 1.0 - 0.95 * np.exp(-0.03353 * pmv**4 - 0.2179 * pmv**2)
142     self._no_occupant = random.random() < (ppd * ppd)
143
144     # 観察
145     obs = self._get_observation()
146
147     # 執務者が退室するか、電池が切れたら報酬 0 で終了
148     if self._no_occupant or self._battery <= 0:
149       return obs, 0, True, False, {}
150
151     # ステップが最大に達した場合には(0.3*残存電力)を報酬に加算して終了
152     elif self.MAX_STEP <= self._current_step:
153       return obs, (1.0 - ppd) + 0.3 * self._battery, True, False, {}
154
155     # その他の場合には執務者の満足度を報酬として出力
```

```
    else:
      return obs, 1.0 - ppd, False, False, {}

  # 環境の状態を描画する（gymnasium 実装）**********
  def render(self):
    print(
      f"Step: {self._current_step}, "
      f"屋外: {self._outdoor_temp:.1f}C, "
      f"屋内: {self._room_temp:.1f}C, "
      f"電池: {self._battery:.1f}kWh"
    )
```

これは空調設備システムの応答を模擬する *HVACEnvironment* クラスである。コードを解説する前にまず、どのような応答を示すのかを確認しておく。

新しいセルを作り、以下のコードを入力して実行する。

プログラムコード 3.2

```
env = HVACEnvironment()
obs, info = env.reset(1) # 乱数シードを 1 に設定
done = False

# 標準化を戻す
ot = obs[0] * env.TEMP_MAX
rt = obs[1] * env.TEMP_MAX
bt = obs[2] * env.BATTERY_MAX

print("HVAC 環境シミュレーション開始")
print("------------------------------")
print(f"初期状態 屋外:{ot:.1f}C 室内:{rt:.1f}C 電池:{bt:.1f}kWh")

terminated = False
total_reward = 0

while not terminated:
  try:
    af_in = float(input("風量[kg/s] (0.2-1.0): "))
    st_in = float(input("給気温度[C] (10.0-40.0): "))

    # 行動を[0, 1]の範囲に正規化
    action_normalized = [
      (af_in - 0.2) / 0.8,
      (st_in - 10.0) / 30.0
    ]

    obs, reward, terminated, truncated, info = env.step(action_normalized)
    total_reward += reward

    env.render()
    print(f"即時報酬: {reward:.2f}, 合計報酬: {total_reward:.2f}")

    if env._no_occupant: print("執務者が退室しました。")
    if env._battery <= 0: print("電池が切れました。")
    if env._current_step >= env.MAX_STEP: print("最大ステップ数に到達しました。")

  except Exception as e:
    print(f"入力エラー: {e}")
    continue

print("エピソード終了")
```

「HVAC 環境シミュレーション開始」というメッセージとともに、外気温度、室内温度、電池の残存量の初期状態が示される（図 3.3）。続けて風量と給気温度の入力が求められるため、風量[kg/s]に 1、給気

温度[℃]に 15 を入力する。*Environment* はこれらの *Action* を受け取り、室内温度と電池の残存量を更新し、報酬を計算する。これらの情報は次のタイムステップの外気温度とともに再び画面に表示される。本例では室温は 24 ℃ に下がり、電池の容量は 20 kWh から 17 kWh に減少する。不満足率は 6%のため、報酬は 0.94 と高い。

```
HVAC 環境シミュレーション開始
------------------------------
初期状態 屋外:35.0C 室内:24.0C 電池:20.0kWh
風量[kg/s] (0.2-1.0): 1
給気温度[C] (10.0-40.0): 15
Step: 1, 屋外: 32.6C, 屋内: 25.0C, 電池: 17.0kWh
即時報酬: 0.94, 合計報酬: 0.94
風量[kg/s] (0.2-1.0):
```

図 3.3　空調 Environment の応答 1

次のステップでは風量を 0.2、給気温度を 40 としてみる（図 3.4）。電力の消費量は少なくなるが、室内の温度は大きく上がり、執務者が退室してしまうため、Episode が終了する。

```
HVAC 環境シミュレーション開始
------------------------------
初期状態 屋外:35.0C 室内:24.0C 電池:20.0kWh
風量[kg/s] (0.2-1.0): 1
給気温度[C] (10.0-40.0): 15
Step: 1, 屋外: 32.6C, 屋内: 25.0C, 電池: 17.0kWh
即時報酬: 0.94, 合計報酬: 0.94
風量[kg/s] (0.2-1.0): 0.2
給気温度[C] (10.0-40.0): 40
Step: 2, 屋外: 32.9C, 屋内: 33.7C, 電池: 16.0kWh
即時報酬: 0.00, 合計報酬: 0.94
執務者が退室しました。
エピソード終了
```

図 3.4　空調 Environment の応答 2

以下、*HVACEnvironment* クラスの内容を解説する。

初期化処理

31~48 行はクラスの初期化処理である。50~56 行で定義するメソッドを呼ぶことで室内の温度や電池の残量などを初期化する。

Observation（38~42 行）の対象は外気温度、室内温度、残電力量の 3 つ、*Action*（44~48 行）の項目は給気風量と給気温度の 2 つである。迷路の例とは異なり、本例では離散値ではなく連続量が必要である。連続量の場合には spaces.Box を使う。ここでそれぞれの値の最小値（low）を 0.0、最大値（high）を 1.0 としていることに注意しなければならない。一般に強化学習の内部で使われるニューラルネットは入出力を 0.0~1.0 に標準化した方が、学習が成功しやすい。

reset メソッド

58~86 行は *reset* メソッドの定義である。本例では迷路の事例とは異なり、引数として与えられた乱数シード（*seed*）を使う。これは以下を目的としている。本例では、外気温度が正規乱数に従うこと（式 3.5）や、執務者が一様乱数の値に応じて退室すること（式 3.8）などにより、計算に不確実性が表現されている。このような不確実性の存在は、*Agent* を様々な入力に対して堅牢に鍛えるためには必要だが、一方で、

Agent の学習の状態を確認する際には邪魔になる。つまり、*Agent* が *Reward* を多く獲得したとしても、たまたま乱数が良かったためなのか、それとも真に *Agenet* の学習が進行したためなのか、判断することが難しくなるためである。このため、通常、学習の進捗を評価する場面では、乱数の出方を固定する。乱数シードに同じ整数を設定すれば乱数の出る順序は確定的になる。このために *seed* が与えられた場合にはその値を乱数処理クラスである random に設定し、*seed* がない場合には現在時刻で乱数シードを設定している（79~83 行）。

86 行で初期の *Observation* の結果を出力する。*Observation* は 88~94 行に定義したメソッドで計算しており、温度と電池容量の最大値（26, 27 行で定数として定義）で割ることで、温度と電池残存容量を 0~1.0 の範囲に標準化していることに注意されたい。

step メソッド

96~157 行は *step* メソッドの定義である。

10 時間の経過で *episode* を強制的に終了させるため、*Environment* は現在の経過時間を保持しなければならない。このため、118 行で *step* の数を加算する。

121~122 行は執務者の *Action* の取得である。*Action* は 0.0~1.0 に標準化されているため、線形変換して風量を 0.2~1.0 kg/s、給気温度を 10~40°C に変換する。

124~142 行は *Environment* の *state* の更新処理である。式 3.9 と 3.10 を使い、消費電力と電池の残存容量を求める（124~127 行）。式 3.2~3.4 を使い、室温を更新する（129~132 行）。式 3.5 に従い、外気温度を確率的に更新する（134~136 行）。式 3.6 と式 3.7 を使って執務者の不満足率を計算し、式 3.8 に従って執務者を確率的に退室させる（138~142 行）。

145 行で *Observation* を計算する。

147~153 行は *episode* の終了判定である。執務者が退室するか、電池が切れた場合（148 行）には、*Observation* に続けて 0 の *Reward* を出力し、終了フラグを True にする。電池が残ったまま、10 時間が経過した場合（152 行）には、電池の残存容量に 0.3 を乗じた値をボーナスとして *Reward* に加算する。この場合も終了フラグは True とする。その他の場合には *step* が継続するため、執務者の満足率を *Reward* とし、終了フラグを False とする（157 行）。

3.2 Agent の訓練プログラムの開発

新しいセルを作り、以下のコードを入力して実行する。

プログラムコード 3.3

```
import gymnasium as gym
import numpy as np
from stable_baselines3 import PPO
from gymnasium.envs.registration import register
import torch.nn as nn
from pathlib import Path
from hvac_env_loader import register_hvac_env, ENV_ID

def evaluate_model(model, env, n_episodes=5):
    """
    指定された強化学習モデルを評価し、1 エピソードにおける累積報酬を返す。
    Parameters:
    -----------
    model : BaseAlgorithm
        学習済みまたは学習中の強化学習モデル（例：PPO インスタンス）
    env : gym.Env
```

```
17          評価対象の環境（Gymnasium 形式）
18      n_episodes : int
19          評価に使用するエピソード数（デフォルト：5）
20      Returns:
21      -------
22      avg_reward : float
23          平均報酬
24      """
25      total_rewards = []
26      for i in range(n_episodes):
27          obs, _ = env.reset(seed=i)  # シードを変えて n 回試す
28          done = False
29          truncated = False
30          episode_reward = 0.0
31          while not (done or truncated):
32              action, _ = model.predict(obs, deterministic=True)
33              obs, reward, done, truncated, _ = env.step(action)
34              episode_reward += reward
35          total_rewards.append(episode_reward)
36
37      avg_reward = sum(total_rewards) / len(total_rewards)
38      return avg_reward
39
40  # 環境の作成
41  register_hvac_env()
42  env = gym.make(ENV_ID)
43
44  # モデルの定義
45  model_file_path = "./model.zip"
46  if Path(model_file_path).exists():
47      print("Loaded an existing model. Resuming training...")
48      model = PPO.load(model_file_path, env=env)
49  else:
50      print("No existing model found. Creating a new model and starting training...")
51      policy_kwargs = dict(
52          net_arch=[32, 16],
53          activation_fn=nn.ReLU,
54      )
55      model = PPO("MlpPolicy", env, policy_kwargs=policy_kwargs, verbose=1, gamma=0.99)
56
57  print(model.policy)
58
59  # 学習
60  rewards = []
61  for n in range(200):
62      print(f"Training iteration {n + 1}...")
63
64      # 学習して保存
65      model.learn(total_timesteps=10*10) # 10h×10 回相当の学習
66      model_file = f"./saved_models/model_{n+1}.zip"
67      model.save(model_file)
68
69      # 報酬の評価
70      evl_rwd = evaluate_model(model, env, n_episodes=5)
71      rewards.append(evl_rwd)
72      np.save("rewards.npy", rewards)
```

このプログラムの骨子は 41~42 行、51~55 行、65 行、の 3 ヶ所である。

41~42 行では、先に開発したプログラムコード 3.1 とプログラムコード 3.2 を使って *HVACEnvironment* のインスタンスを作り、変数 env に代入している。

55 行は学習の方法の指定である。*Agent* を学習させる方法は 1 つだけではなく、stable_baselines3 は PPO、A2C、DQN など、多くの学習方法に対応している。本例では Proximal Policy Optimization（PPO）を使う。第 1 引数（*MlpPolicy*）で、多層ニューラルネットワーク（MLP: Multi Layer Perceptron）を使うことを指

示し、その具体的な構造は第 3 引数（*policy_kwargs*）で与える。この構造は 51~54 行で設定しており、ここでは中間層が 2 層でそれぞれの素子数が 32 と 16 のニューラルネットとし､伝達関数に ReLU を使う。第 2 引数は *Environment* の指定で、ここでは先に用意した *HVACEnvironment* を与える。第 4 引数の verbose は学習過程をどの程度、画面表示するかの設定で、学習結果には影響が無いのでとりあえず 1 としておけば良い。第 5 引数の gamma は割引率である。

65 行は学習の実行命令で、total_timesteps で学習回数を指定する。ここでは 10 時間が 10 回分とし、100 step の学習を指示している。

上記以外の行は補助的な処理のため、概略のみを以下に示す。

9~38 行は、*Agent* の性能を評価するためのメソッドである。引数で与えられた数だけ、*episode* を繰り返して平均的な *Reward* を求めて出力する。

44~55 行は学習モデルの作成処理で、新規作成の場合の作成方法（50~55 行）は上に述べたとおりである。この他に、過去に学習して保存しておいたデータ（model.zip）がある場合には、それを読み込んで途中から学習を再開することもできる（46~48 行)。

60 行以降が学習処理で、100 回（10 時間×10 回分）の学習を全部で 200 回繰り返す。つまり、合計では 20,000 回の学習が行われる。100 回ずつを 200 回に分割したのは途中経過を確認することが目的である。66, 67 行で途中まで学習したモデルの構造を model.zip に書き出す。70 行で学習中のモデルの性能を評価し、得られた平均 *Reward* を rewards.npy に書き出す。この rewards.npy をグラフ化すれば、学習の進展に応じてどのように *Reward* が拡大したのかを確認できる。

3.3 学習の実行

1) 基本の学習

Colab の新規セルに次のコードを入力して *train.py* を起動する。

```
!python train.py
```

図 3.5 に示すように 100 step の学習が終わるたびにその経過が表示される。ep_len_mean と ep_rew_mean は 1 つの *episode* の平均的な *step* 数と *Reward* である。うまく空調を制御できれば執務者が部屋にとどまり、*step* 数は長く、報酬は高くなるため、学習が進むにつれてこれらの値は大きくなる。

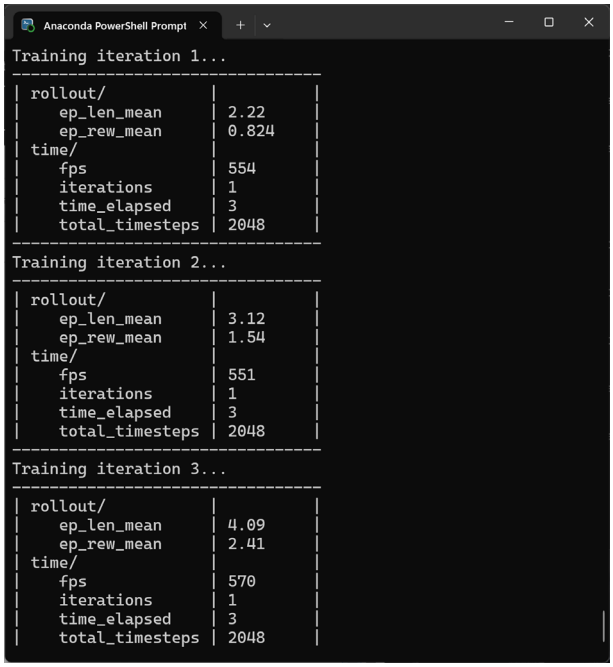


図 3.5　学習の途中経過

200 回の学習を終えたら、次のコマンドを入力して学習の経過を確認する。

!python drawchart.py

図 3.6 に示すように、学習回数が増えるにつれ、*Reward* が高まり、*Agent* の性能が上がっていることがわかる。ただし、学習には乱数を使うため、毎回、この図と完全に一致する結果が得られるとは限らない。

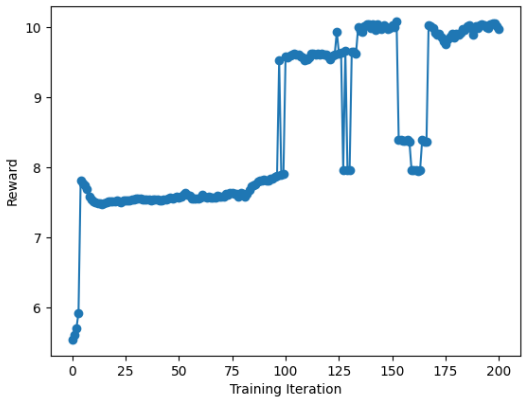


図 3.6　学習回数と報酬の関係

具体的にどのような制御をする *Agent* が得られたのかを確認する。saved_models ディレクトリには model_*n*.zip というモデル構造ファイルが書き出される。従って、200 回の学習が終わると、model_1.zip から model_200.zip まで、200 個のモデルが並ぶ。model_1.zip は 1 回しか学習をしていないほぼ初期状態のモデル、model_200.zip は十分に学習を積んだ最終のモデルである。そこで、これらのモデルを使った場

合の空調制御の結果を確認する。セルに次のコードを入力すると保存されたモデルで *Agent* が初期化されて、1 回の *episode* の計算結果がグラフ化される（図 3.7 と図 3.8）。

```
!python verify.py saved_models/model_1.zip
!python verify.py saved_models/model_200.zip
```

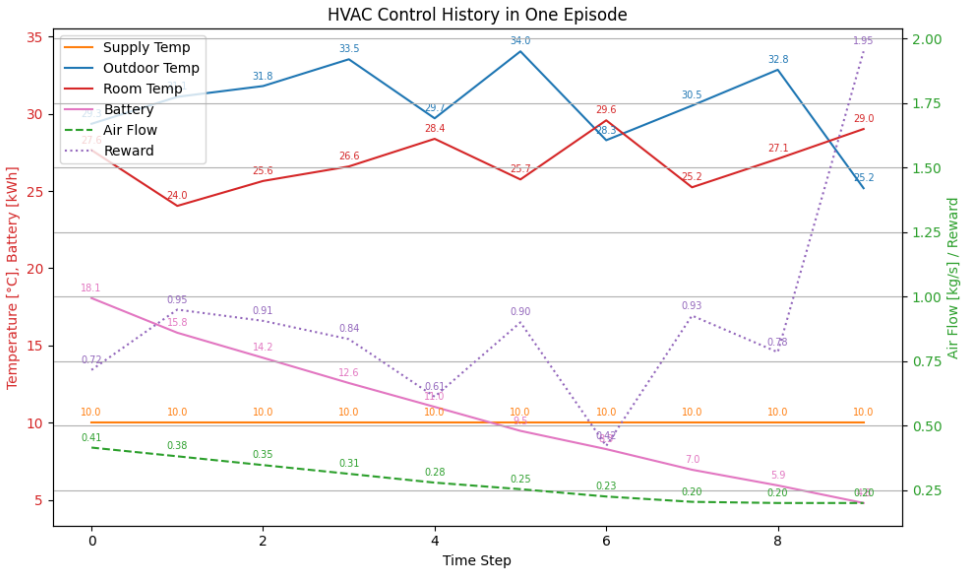


図 3.7　Agent の空調制御結果（model_1.zip）

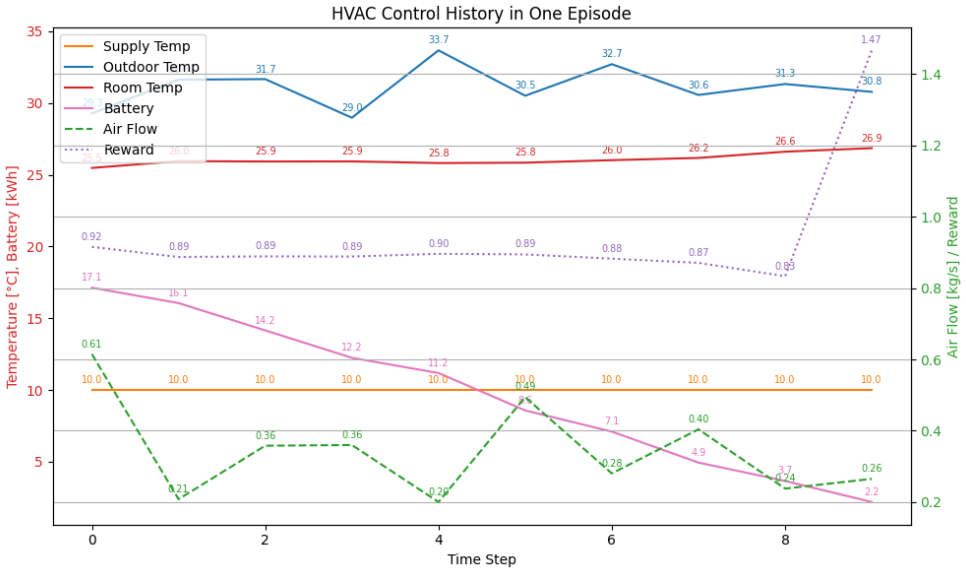


図 3.8　Agent の空調制御結果（model_200.zip）

ほぼ学習をしていない図 3.7 では室温が上下に大きく揺れ、安定していない。かろうじて 10 時間の運用はできたものの、4 時間目と 6 時間目の不満足率は高く、執務者が退室する可能性があり、危険な運用

である。一方で十分に学習した図 3.8 では風量の制御によって室温がほぼ一定に保たれている。

ここで、室温が 26 ℃ 程度に維持されている点には注意が必要だろう。図 3.2 に示したように、このモデルでは室温が 24 ℃ 程度のときに不満足率は最小化するため、執務者の退室リスクをおさえるという観点からは 24 ℃ 程度に室温を維持すべきである。しかし、この *Agent* はそれよりも高い温度を選択している。

これは最終ステップにおいて電池の残存容量によるボーナスを獲得することを狙っているためであろう。すなわち、室温をやや高めに維持することで、執務者の退室リスクを少し上げることの代償に、エネルギーの消費を抑えることが最適と学習したということである。この点を確認するため、*Reward* の計算法などを変更して再学習してみる。

2) 報酬関数の変更

電力は使い放題であるという前提で問題を再設定する。このためプログラムコード 3.4 に示すように *HVACEnvironment* の *step* メソッドを 2 ヶ所、変更する。1 つは *step* の冒頭で、電池残量を毎回満充電させる。もう 1 つは *Reward* の計算で、最終タイムステップの場合に電池の残量ボーナスを削除する。以上の変更により、*Agent* にとって省エネルギーな運転を心がけて電池の残量を維持する利益はなくなった。

プログラムコード 3.4

HVACEnvironment.py

```
        ⋮
# ステップ追加
self._current_step += 1
self._battery = self.BATTERY_MAX # 毎事満充電
        ⋮
        ⋮
elif self.MAX_STEP <= self._current_step:
    # return obs, (1.0 - ppd) + 0.3 * self._battery, True, False, {}
    return obs, 1.0 - ppd, True, False, {}
        ⋮
```

変更したプログラムを使って再度学習した結果を図 3.9 に示す。室温は先の例に比べて低く、24 ℃ 前後に維持されている。これによって、満足率もほぼ最高である 0.95 程度に維持されており、快適性を最大化する制御を学習できている。

このようにどのように *Reward* を計算するのかという、報酬関数の設計は *Agent* の性能を大きく左右する。重要なことは、我々は *Reward* を使って *Agent* に手段ではなく目標を伝えるべきだという点である。例えば本例で満足率を *Reward* にせず、室温と 24 ℃ の近さを *Reward* にした場合、これは手段を伝えたことになる。つまり「24 ℃ は満足度が高い」という我々の知見を使うことで「満足度を高めるために 24 ℃ に近づける」という手段を伝えている。本来、*Agent* は試行錯誤することで目標を満たす様々手段を探索すべきであり、早々に 24 ℃ という手段を与えてしまっては、その他のあり得るべき手段が発見される可能性がなくなってしまう。

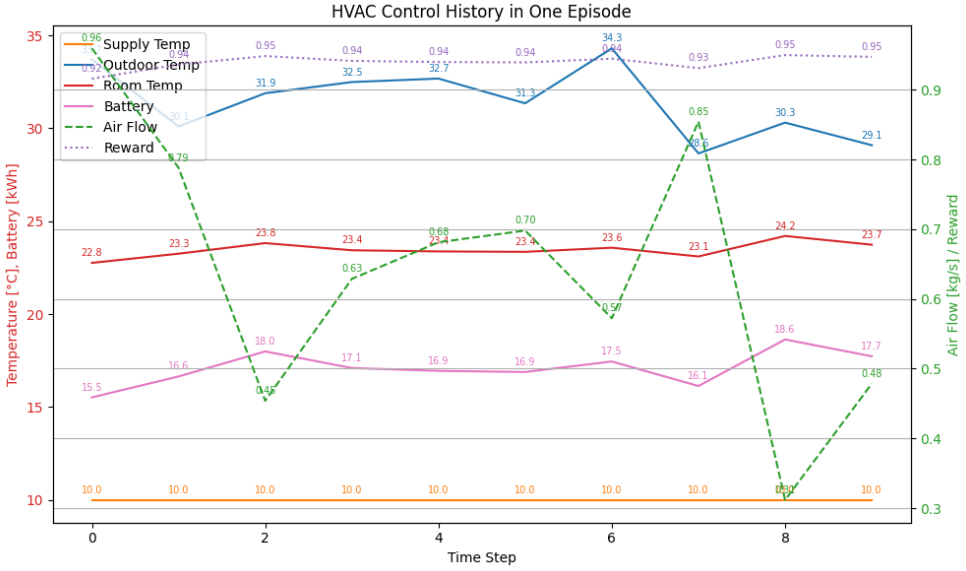


図 3.9　Agent の空調制御結果

4. 空調設備システムの最適化 2（実システムへの展開に向けて）

前章では空調設備システムのモデルと強化学習と、両方を 1 つのプログラムで実装した。強化学習の性能に関わる研究であればこれでも良いが、現実の建物ではこのような構成は許されない。建築設備システムは他者が構築したものであり、現実時間で稼働している。このようなシステムから動的に情報を取り出し、強化学習を実行する技術が必要になる。

建築設備分野で使われているオープンな通信方式として BACnet がある。この BACnet 通信を使えば実システムと連携がしやすく、建築設備システムの現在の状態を受信し（強化学習における *Observation*）、次のタイムステップにおける制御を送信する（強化学習における *Action*）ことができる。

そこで本章では、BACnet 通信に対応した精緻なシミュレーションプログラム（以降、エミュレータと呼ぶ）を導入し、これに対して外部から BACnet 通信で強化学習を適用する方法を解説する。

4.1 エミュレータと強化学習キットの内容

1) 強化学習キットの導入

本エミュレータは空気調和・衛生工学会で開発されたもので、以下の Web サイトで最新版が公開されている。

https://www.wccbo.org

https://github.com/et0614/shizuku

git にアクセスして、強化学習の試験を目的に開発された「Shizuku2-RL-kit.zip」をダウンロードする。参考に、この圧縮ファイルの中身を図 4.1 に示す。

```
Shizuku2-RL-kit.zip
        ├Shizuku2/
        │      ├Shizuku2.exe
        │      └その他
        ├szlib
        ├AbstractShizuku2Environment.py
        ├Shizuku2EnvironmentSample.py
        └その他
```

図 4.1　Shizuku2-RL-kit.zip の中身

Shizuku2 ディレクトリの中にある Shizuku2.exe がエミュレータである。個別分散型パッケージが導入された 1000 m^2 程度のオフィスビルがモデル化されており、夏季または冬季の 1 週間の計算ができる。詳細については同封のレファレンスを参照されたい。

szlib ディレクトリの中には python を使ってエミュレータと BACnet で通信するためのライブラリが入っている。通信の仕様についてもレファレンスを参照されたい。

AbstractShizuku2Environment.py と Shizuku2EnvironmentSample.py が強化学習のための python プログラムである。実システムやエミュレータを対象に強化学習で最適化をするときには、大きく 2 つの処理が必要になる。1 つはシステムとの通信、もう 1 つは強化学習である。より優れた結果を得るために強化学習は様々な手法を試したいが、システムとの通信自体は共通の処理で良い。そこで、2 つの機能を親子のクラスに分離した。図 4.2 にクラスの役割分担を示す。

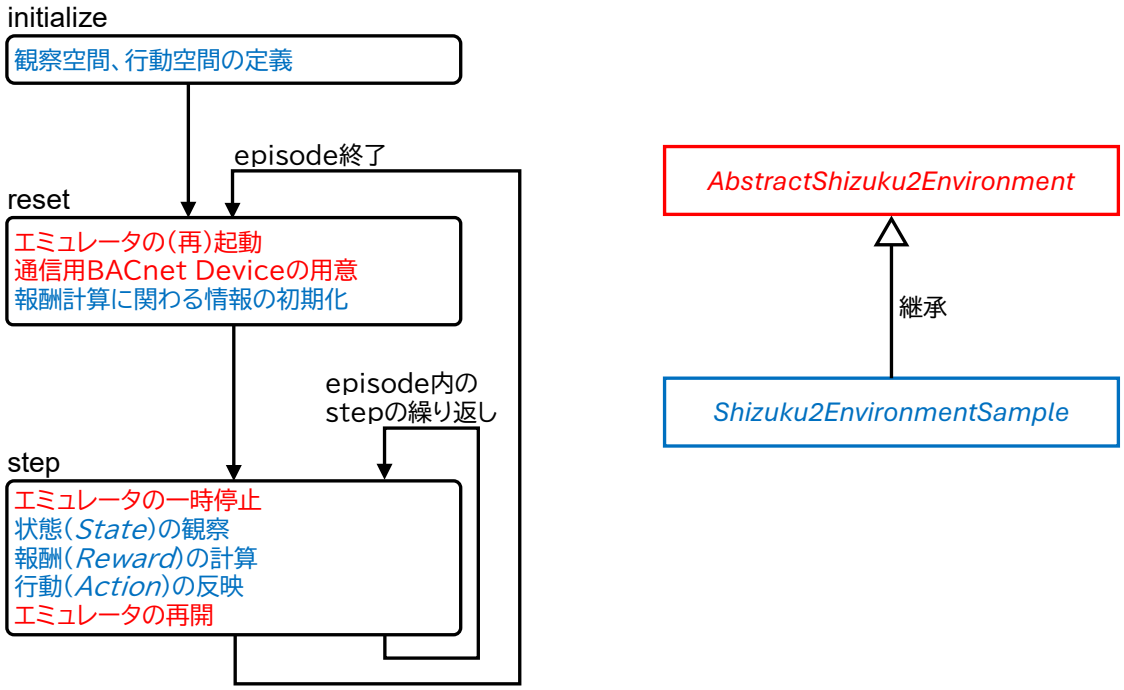


図 4.2　クラスの役割分担

AbstractShizuku2Environment は抽象クラスで、赤字で示した、エミュレータの起動、通信のための準備、*step* ごとのエミュレータの一時停止と再開、を担当する。実システムであれば、強化学習を実施する時刻までの待機などが担当すべき処理になる。これを継承した派生クラスが *Shizuku2EnvironmentSample* で、青字で示した、強化学習に関わる処理を担当する。

AbstractShizuku2Environment はエミュレータをバックグラウンドで動かしながら適宜、BACnet で通信する非同期で複雑なプログラムで、強化学習の中心的な話題ではないため、その説明は省略する。以下では派生クラスである *Shizuku2EnvironmentSample* について解説する。

2) 環境クラス（Shizuku2EnvironmentSample）

Shizuku2EnvironmentSample をプログラムコード 4.1 に示す。なお、名前に *Sample* がついている通り、これは一例に過ぎない。これはこのエミュレータが持つすべての制御項目を網羅的に強化学習の対象項目にするサンプルである。

プログラムコード 4.1

```
 1 import numpy as np
 2 import csv
 3 from scipy.stats import norm
 4 from gymnasium.spaces import Box, MultiDiscrete
 5 from AbstractShizuku2Environment import AbstractShizuku2Environment
 6 from szklib import VRFSystemCommunicator as vrc
 7 from szklib import OccupantCommunicator as occ
 8 from szklib import VentilationSystemCommunicator as vsc
 9
10 IUNIT_NUM = 5+4+5+4 # 室内機の数
11
12 class Shizuku2EnvironmentSample(AbstractShizuku2Environment):
13     """ Shizuku2 の環境クラスサンプル """
14
15     # region 初期化処理
```

```
16
17      def __init__(self, action_timestep:int=1800, backup_interval:int=100, eval_mode=False):
18          super(Shizuku2EnvironmentSample, self).__init__(action_timestep, backup_interval, eval_mode)
19
20          # インスタンス変数の定義
21          self._prevRtn = [] # 1タイムステップ前の出力
22
23          # 成績書き出し用の空のcsvファイルを用意
24          open('eci.csv', 'w').close()
25
26      # endregion
27
28      # region gymnasiumのEnv実装
29
30      def reset(self, seed=None, options=None):
31          self.energy = 0
32          self.disrate = 0
33          self.ttl_rwd = 0
34          self.prev_rwd = 0
35          return super().reset(seed, options)
36
37
38      def step(self, action):
39          return super().step(action)
40
41      def render(self, mode='human'):
42          super().render(mode)
43
44      # endregion
45
46      # region 行動・観測・報酬の定義
47
48      def _define_space(self):
49          """
50          行動空間と状態空間を定義する
51          """
52          # 行動空間の定義
53          md = []
54          i_unit_num = [5,4,5,4] # 各系統の室内機の台数
55          # 室外機の制御項目
56          for i in range(len(i_unit_num)):
57              md.extend([
58                  3,  # モード 0:heating, 1:cooling, 2:fan
59                  11, # 蒸発温度 y=x+3
60                  11 # 凝縮温度 y=x+40
61                  ])
62              # 室内機の制御項目
63              for _ in range(i_unit_num[i]):
64                  md.extend([
65                      2,  # 発停 0:off, 1:on
66                      31, # 室温設定値 y=22+0.2x
67                      3,  # ファン風量 0:Low, 1:Middle, 2:High
68                      5,  # ファン風向 0:Horizontal, 1:22.5deg, 2:45.0deg, 3:67.5deg, 4:Vertical
69                      2,  # 執務者リモコン操作許可 0:Disabled, 1:Enabled
70                      2,  # 全熱交換器発停 0:off, 1:on
71                      2,  # 全熱交換器バイパス制御 0:Disabled, 1:Enabled
72                      3   # 全熱交換器ファン風量 0:Low, 1:Middle, 2:High
73                      ])
74
75          self.action_space = MultiDiscrete(md)
76
77          # 状態空間の定義 (曜日+時刻+外気温度+外気湿度+ゾーン数x(ゾーン温度+ゾーン執務者数))
78          self.observation_space = Box(low=0.0, high=1.0, shape=(4 + 2 * IUNIT_NUM,), dtype=float)
79          self.state = np.zeros(4 + 2 * IUNIT_NUM, dtype=np.float64)  # 初期状態はすべて0
80          self.goal_state = None  # 目標状態は無し
81
82      async def _perform_action(self, action):
83          """
84          行動をエミュレータに反映する
```

```
        Args:
            action: 行動
        Returns:
            None
        """

        # 室外機の制御項目
        indx:int = 0
        i_unit_num = [5,4,5,4] # 各系統の室内機の台数
        for i in range(len(i_unit_num)):
            match action[indx]:
                case 0: mode = vrc.Mode.Heating
                case 1: mode = vrc.Mode.Cooling
                case _: mode = vrc.Mode.ThermoOff
            indx+=1
            await self.vrfCom.change_evaporating_temperature(i+1,3 + action[indx])
            indx+=1
            await self.vrfCom.change_condensing_temperature(i+1,40 + action[indx])
            indx+=1
            for j in range(i_unit_num[i]):
                is_on = action[indx] == 1
                indx+=1

                # VRF 室内機操作
                if is_on:
                    # 制御内容を取得
                    sp_tmp = 22 + action[indx] * 0.2
                    indx+=1
                    match action[indx]:
                        case 0: fan_spd = vrc.FanSpeed.Low
                        case 1: fan_spd = vrc.FanSpeed.Middle
                        case _: fan_spd = vrc.FanSpeed.High
                    indx+=1
                    match action[indx]:
                        case 0: fan_dir = vrc.Direction.Horizontal
                        case 1: fan_dir = vrc.Direction.Degree_225
                        case 2: fan_dir = vrc.Direction.Degree_450
                        case 3: fan_dir = vrc.Direction.Degree_675
                        case _: fan_dir = vrc.Direction.Vertical
                    indx+=1
                    usr_pmt = action[indx] == 1
                    indx+=1

                    # 制御を反映
                    await self.vrfCom.turn_on(i+1,j+1) # 空調運転
                    await self.vrfCom.change_mode(i+1,j+1,mode)
                    await self.vrfCom.change_fan_speed(i+1,j+1,fan_spd)
                    await self.vrfCom.change_direction(i+1,j+1,fan_dir)
                    if(usr_pmt):
                        await self.vrfCom.permit_local_control(i+1,j+1)
                    else:
                        await self.vrfCom.prohibit_local_control(i+1,j+1)
                        await self.vrfCom.change_setpoint_temperature(i+1,j+1,sp_tmp)
                else:
                    indx+=4
                    await self.vrfCom.turn_off(i+1,j+1) # 空調停止

                # 全熱交操作
                hex_on = action[indx] == 1
                indx+=1
                if hex_on:
                    # 制御内容を取得
                    hex_byps = action[indx] == 1
                    indx+=1
                    match action[indx]:
                        case 0: hex_spd = vsc.FanSpeed.Low
                        case 1: hex_spd = vsc.FanSpeed.Middle
                        case _: hex_spd = vsc.FanSpeed.High
                    indx+=1
```

```
154
155                     # 制御を反映
156                     await self.vscCom.start_ventilation(i+1,j+1)
157                     await self.vscCom.change_fan_speed(i+1,j+1,hex_spd)
158                     if hex_byps:
159                         await self.vscCom.enable_bypass_control(i+1,j+1)
160                     else:
161                         await self.vscCom.disable_bypass_control(i+1,j+1)
162                 else:
163                     indx+=2
164                     await self.vscCom.stop_ventilation(i+1,j+1)
165
166     async def _observe_state(self):
167         """
168         状態を観察する
169         Args:
170             None
171         Returns:
172             観察結果
173         """
174         indx:int = 0
175         rtnList = []
176
177         # 1: 曜日
178         rtnList.append(self.CurrentDateTime.weekday() / 6)
179         indx += 1
180
181         # 2: 時刻
182         now = self.CurrentDateTime
183         seconds_since_midnight = now.hour * 3600 + now.minute * 60 + now.second # sec で表示
184         rtnList.append(seconds_since_midnight / 86400)
185         indx += 1
186
187         # 3: 外気乾球温度
188         val = await self.envCom.get_drybulb_temperature()
189         rtnList.append(val[1] / 40 if val[0] else self._prevRtn[indx])
190         indx += 1
191
192         # 4: 外気相対湿度
193         val = await self.envCom.get_relative_humidity()
194         rtnList.append(val[1] / 100 if val[0] else self._prevRtn[indx])
195         indx += 1
196
197         # 各ゾーンの温度
198         i_unit_num = [5,4,5,4] # 各系統の室内機の台数
199         for i in range(len(i_unit_num)):
200             for j in range(i_unit_num[i]):
201                 val = await self.envCom.get_zone_drybulb_temperature(i+1,j+1)
202                 rtnList.append(val[1] / 40 if val[0] else self._prevRtn[indx])
203                 indx += 1
204
205         # 南側ゾーンの執務者数
206         for i in range(1, 10):
207             val = await self.occCom.get_zone_occupant_number(occ.Tenant.South, i)
208             rtnList.append(val[1] / 10 if val[0] else self._prevRtn[indx])
209             indx += 1
210             val = await self.occCom.get_zone_occupant_number(occ.Tenant.North, i)
211             rtnList.append(val[1] / 10 if val[0] else self._prevRtn[indx])
212             indx += 1
213
214         self._prevRtn = rtnList # 出力を保存
215         return np.array(rtnList)
216
217     async def _calculate_reward(self, is_final_step):
218         """
219         報酬を計算する
220         Args:
221             is_final_step: エピソードの最終ステップか否か
222         Returns:
223
```

```
224             報酬
225         """
226         # 不満足者数 [人] (最大で 100 人程度, 基準乱数で 86 人)
227         while not (result:= await self.occCom.get_occupant_number(self.occCom.Tenant.South))[0]:pass
228         dis_occ = result[1]
229         while not (result:= await self.occCom.get_occupant_number(self.occCom.Tenant.North))[0]:pass
230         dis_occ += result[1]
231         while not (result:= await self.envCom.get_instantaneous_dissatisfaction_rate())[0]:pass
232         dis_occ *= result[1]
233 
234         # エネルギー消費量 [kW] (最大で 35kW 程度, 平均で 8kW 程度)
235         while not (result:= await self.vrfCom.get_outdoor_unit_electricity(1))[0]:pass
236         vrf_elec = result[1]
237         while not (result:= await self.vrfCom.get_outdoor_unit_electricity(2))[0]:pass
238         vrf_elec += result[1]
239         while not (result:= await self.vrfCom.get_outdoor_unit_electricity(3))[0]:pass
240         vrf_elec += result[1]
241         while not (result:= await self.vrfCom.get_outdoor_unit_electricity(4))[0]:pass
242         vrf_elec += result[1]
243 
244         # 不満足者数とエネルギー消費を重み付け平均
245         rwd = -(2.0 * dis_occ / 86.0 + 1.0 * vrf_elec / 35.0) # ほぼ 0-1 の範囲にした上で重み付け平均
246         if self.prev_rwd != 0:
247             rwd = rwd + (rwd - self.prev_rwd) # Shaping: 前回の報酬との差分を加算
248 
249         self.ttl_rwd = self.ttl_rwd + rwd
250 
251         if is_final_step:
252             with open('eci.csv', 'a', newline='') as f:
253                 while not (result:= await self.envCom.get_averaged_dissatisfaction_rate())[0]:pass
254                 self.disrate = result[1]
255                 while not (result:= await self.envCom.get_total_energy_consumption())[0]:pass
256                 self.energy = 0.001 * result[1]
257                 writer = csv.writer(f)
258                 is_summer = (self._final_dtime.month == 7)
259                 eci = self._calc_eci(self.energy, self.disrate, is_summer)
260                 writer.writerow([self.energy, self.disrate, self.ttl_rwd, eci])
261         return rwd
262 
263     @staticmethod
264     def _calc_eci(energy, disrate, is_summer):
265         if is_summer:
266             lmd_e = 0.8678
267             myu_e = 3.9653
268             sig_e = 1.5129
269             lmd_d = -0.4356
270             myu_d = -2.4076
271             sig_d = 0.3343
272         else :
273             lmd_e = 0.3899
274             myu_e = 0.8598
275             sig_e = 0.8777
276             lmd_d = -1.3428
277             myu_d = -5.4246
278             sig_d = 1.0083
279 
280         pe = 1 - norm.cdf(((energy ** lmd_e - 1) / lmd_e - myu_e) / sig_e)
281         pd = 1 - norm.cdf(((disrate ** lmd_d - 1) / lmd_d - myu_d) / sig_d)
282         return np.sqrt(pe * pd)
283 
284     # endregion
```

12 行で *AbstractShizuku2Environment* を継承する。この継承に加え、28~44 行にあるように step, reset, render で親クラスの処理を呼び出すことで、BACnet 通信関連の処理を親クラスに任せられる。

17~24 行は初期化処理である。BACnet 通信失敗に備えた変数（_prevRtn）を確保し、途中経過書き出し

のための csv ファイルを用意する。詳細は後述する。

28~44 行は *Env* の実装である。本来は図 4.2 に示したようにエミュレータと BACnet 通信をしながら強化学習を実行するという複雑な処理が必要になるが、この大きな処理の流れは既に親クラスで実装済みである。従って、ここで追加すべき処理はほとんど無い。その代わりに 48~285 行が重要で、ここで強化学習における *Action*, *Observation*, *Reward* を定義する。ここで定義されたメソッドが適宜、親クラスの *step* および *reset* メソッドによって呼び出されることで、強化学習が実行される。

48~80 行は、*Action* 空間と *State*（*Observation*）空間の定義である。*Action* 空間は個別分散型空調機と換気設備に関わる 138 項目、*State*（*Observation*）空間は 40 項目である。それぞれの項目が何を意味しているのかについてはエミュレータのレファレンスを参照されたい。本例では、*Action* は多次元の離散値（MultiDiscrete）、*State* は多次元の実数（Box: float）で表現される。

82~164 行は、*Action* をエミュレータに反映する処理である。action 配列には *Action* が保存されており、この値に応じて制御の値を設定している。

166~216 行は、*State* を *Observation* するための処理である。BACnet で各種の *State* を収集し、これを 0.0~1.0 の範囲に丸めながら出力配列を作る。

218~268 行は、*Reward* の計算である。BACnet で不満足者人数とエネルギー消費量を取得し、これを重み付け平均して *Reward* とする。

3) 強化学習実行プログラム

上記で作成した環境クラスを使って強化学習を実行するプログラムが main.py である。内容をプログラムコード 4.2 に示す。

プログラムコード 4.2

```
 1 import gymnasium as gym
 2 import numpy as np
 3 import torch.nn as nn
 4 import os
 5 from stable_baselines3 import PPO
 6 from gymnasium.envs.registration import register
 7 from stable_baselines3.common.callbacks import BaseCallback
 8
 9 OUTPUT_INTERVAL = 100 #データ書き出し間隔
10 NO_LEARNING = False
11
12 # カスタムコールバッククラス
13 class CustomSaveCallback(BaseCallback):
14
15     def __init__(self, save_freq, save_path, verbose=0):
16         super(CustomSaveCallback, self).__init__(verbose)
17         self.save_freq = save_freq
18         self.save_path = save_path
19         os.makedirs(self.save_path, exist_ok=True)
20         self.reward = 0
21         self.n_epsd = 0
22         self.hist_rwds = [] # 学習過程の記録
23         self.episode_actions = []
24
25     def _on_step(self) -> bool:
26         # 報酬を積算
27         self.reward += self.locals["rewards"][0]
28         if self.n_epsd % OUTPUT_INTERVAL == 0:
29             self.episode_actions.append(np.array(self.locals["actions"]))
30
31         # エピソード終了時
```

```
        if self.locals["dones"][0]:
            if self.n_epsd % OUTPUT_INTERVAL == 0:
                # モデルを保存
                model_file = os.path.join(self.save_path, f"model_{self.n_epsd}.zip")
                self.model.save(model_file)
                # 行動を保存
                actions_array = np.array(self.episode_actions)
                self.episode_actions = []
                action_file = os.path.join(self.save_path, f"actions_{self.n_epsd}.npy")
                np.save(action_file, actions_array)

            print(f"Episode finished with total reward: {self.reward:.2f}")
            # 報酬を保存
            self.hist_rwds.append(self.reward)
            np.save(os.path.join(self.save_path, f"rewards.npy"), self.hist_rwds)
            self.reward = 0

            self.n_epsd += 1 # episode 数をインクリメント

        return True

# ここから学習処理
model_file_path = "./model.zip"

# 環境の作成
TARGET = "Shizuku2EnvironmentSample"
register(
    id=TARGET,
    entry_point=f"{TARGET}:{TARGET}",
)
env = gym.make(
    TARGET,
    backup_interval=OUTPUT_INTERVAL,
    eval_mode=NO_LEARNING)

# モデルの定義
if os.path.exists(model_file_path):
    print("Loaded an existing model. Resuming training...")
    model = PPO.load(model_file_path, env=env)
else:
    print("No existing model found. Creating a new model and starting training...")
    policy_kwargs = dict(
        net_arch=[512, 256, 128, 64],
        activation_fn=nn.ReLU,
    )
    model = PPO("MlpPolicy", env, policy_kwargs=policy_kwargs, verbose=1, gamma=1.0)

print(model.policy)

# 学習
if not NO_LEARNING:
    print("Starting reinforcement learning...")
    custom_callback = CustomSaveCallback(save_freq=1, save_path="./saved_models/")
    model.learn(total_timesteps=48*7*10000, callback=custom_callback)
    print("Reinforcement learning completed.")
# 結果確認
else:
    print("Starting evaluation run...")
    obs, info = env.reset()

    terminated = False
    truncated = False
    total_reward = 0.0

    while not terminated and not truncated:
        # 決定論的な行動を選択
        action, _states = model.predict(obs, deterministic=True)
        # 環境を１ステップ進める
        obs, reward, terminated, truncated, info = env.step(action)
```

```
101         total_reward += reward
102         if terminated or truncated:
103             print(f"Evaluation finished. Total Reward: {total_reward}")
104             break
105 
106     print("Evaluation run completed.")
```

3 章までの例とは異なり、問題が複雑なため、学習にはかなりの時間がかかる。このため、不慮のエラーによる計算中断などに備えて、途中まで学習したモデルのデータを定期的に書き出す。この書き出し間隔が 9 行の OUTPUT_INTERVAL である。また、10 行の NO_LEARNING を True にすれば、学習済みのモデルを使って学習結果を確認することができる。

学習過程で定期的に学習の進捗状況や学習済みモデルを保存するためには、13~51 行に示すように *BaseCallback* の派生クラスを定義する。25~49 行に示すように *step* のたびに実行したい処理を定義する。本例では一定の *episode* 間隔ごとにモデルと構造を保存し（33~41 行）、*episode* が終わるごとに *Reward* の値を保存している（45~47 行）。同じディレクトリにある drawchart.py を使えば、*Reward* が書き出された rewards.npy をグラフ化できるため、途中で学習の進捗が確認できる。

54 行以降は学習処理で、前章の例とほぼ変わらない。ただし、問題の複雑さが大きくなっているため、ニューラルネットは多層化して素子数も増やした（75 行）。

4.2 Colab 環境での実行方法

これまでに解説したシステムを Colab で実行する。

エミュレータは C#言語で構築しており、.NET 環境で動くため、.NET のランタイムを導入する必要がある。また、エミュレータとの間で BACnet 通信するためのライブラリも導入しなければならない。このため、新規のノートブックを作り、セルに以下のコードを入力して実行する。

```
# .NET 8 SDK をインストール
!wget https://dot.net/v1/dotnet-install.sh -O dotnet-install.sh
!chmod +x ./dotnet-install.sh
!./dotnet-install.sh --version latest

#  環境変数に dotnet のパスを追加
import os
os.environ['PATH'] = f"{os.environ['HOME']}/.dotnet:{os.environ['PATH']}"

# BACnet 通信用ライブラリを導入
!pip install bacpypes3

#  強化学習用ライブラリを導入
!pip install stable_baselines3
```

この処理はセッションを起動するたびに 1 回だけ実行すれば良い。

ブラウザの左側にある「ファイル」を開き、Shizuku2-RL-kit.zip をアップロードする（図 4.3）。

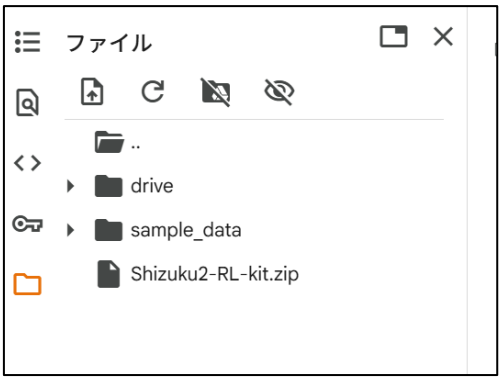


図 4.3　「Shizuku2-RL-kit.zip」のアップロード

新しいセルを開いて以下のコードを実行すれば、アップロードされた zip ファイルが解凍される。

```
!unzip Shizuku2-RL-kit.zip
```

このフォルダのファイルはランタイムが解除されると初期化されるため、アップロードと解凍はそのたびに繰り返す必要がある。

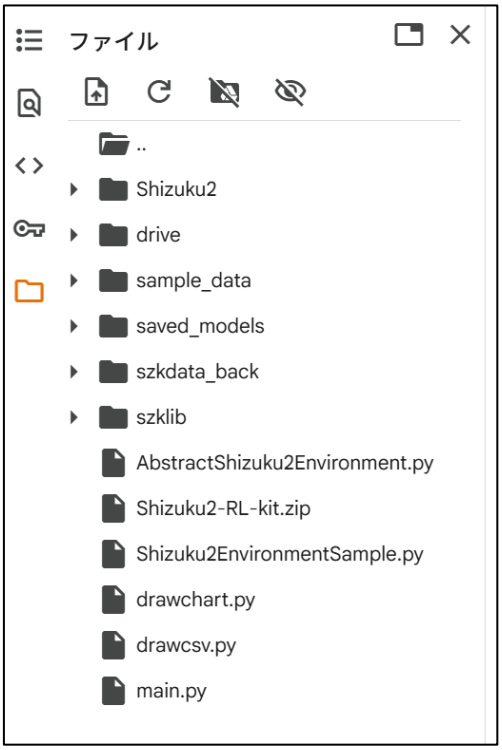


図 4.4　解凍された「Shizuku2-RL-kit.zip」

新しいセルを開き、以下のコードを実行すれば学習が始まる。

```
!python main.py
```

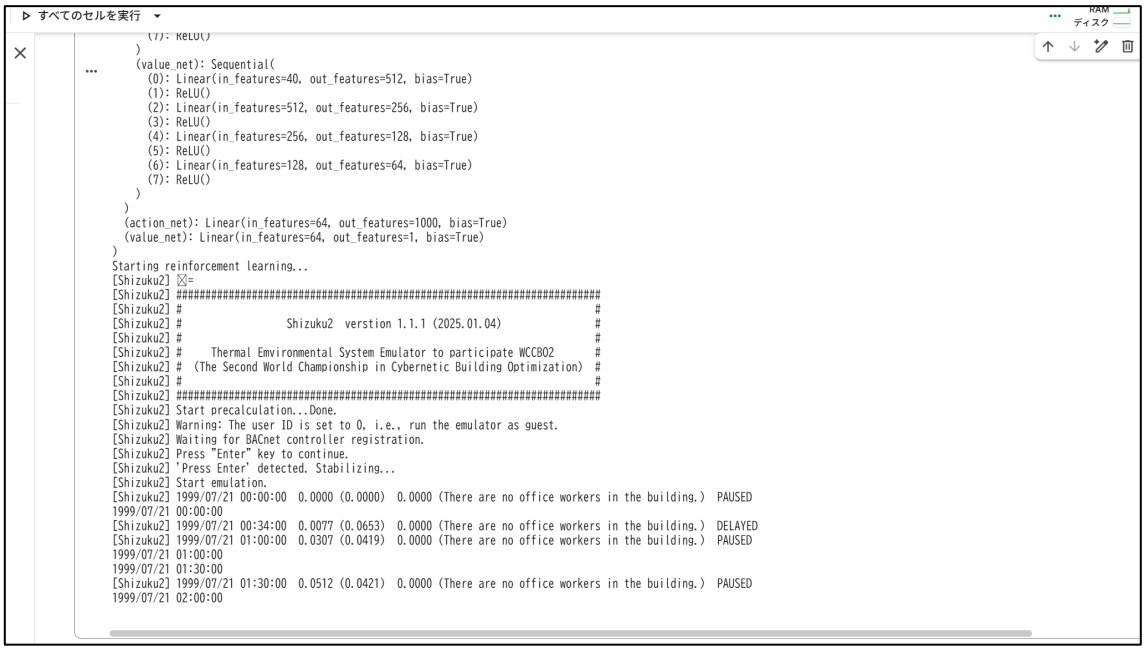


図 4.5　エミュレータを使った強化学習の進捗

本例の問題は複雑で、ある程度の優れた制御を学習するまでに、数日間の学習が必要である。Colab は無料版の場合には数時間の連続実行やブラウザの閉鎖によって計算が中断されてしまう。従って、十分な学習結果を得るためには、有償契約してバックグラウンドで学習が進むようにする方が良い。本例で用意した python プログラムとエミュレータはローカルでも動作するため、GPU 性能の高い PC がある場合には、ローカル環境で実行するという方法もある。

Result.txt にはエミュレータによる出力が保存されており、エネルギー消費量（Energy consumption[GJ]）と平均不満足者率（Average dissatisfied rate[-]）を読み取ることができる。また、エミュレータに同封されている「Benchmark.xlsx」には、多くの技術者がこのエミュレータを使って運用の最適化を図った選手権（WCCBO: World Championship in Cybernetic Building Optimization[2)]）の結果が保存されている。このファイルにエネルギー消費量と平均不満足者率を入力すれば、それがどの程度の成績なのかを計算することができる。図 4.6 は約 20 万回の *episode* を繰り返して学習させたあとに成績を評価した例である。選手権に参加していたとすれば第 7 位の成績である。

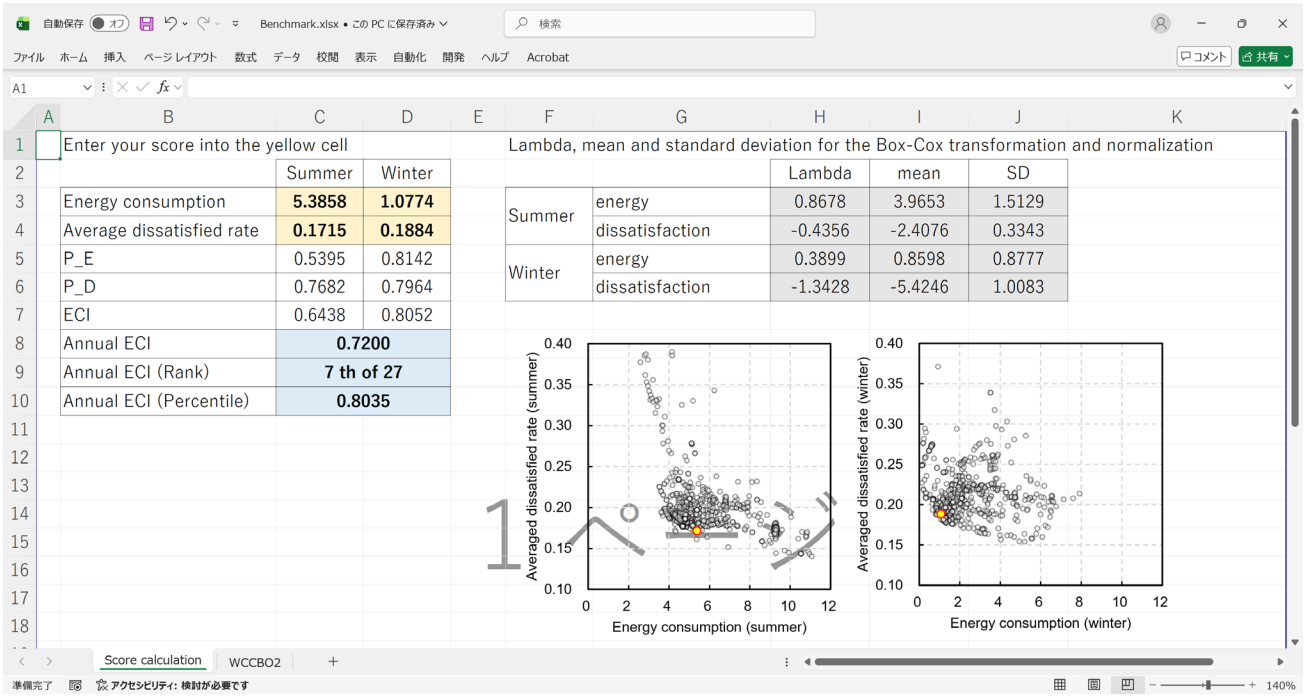


図 4.6　強化学習によるエミュレータの最適化結果

【参考文献】

1）Sutton, R. S., & Barto, A. G. (2018). Reinforcement learning: An introduction (2nd ed.). MIT Press.

2）Togashi, E., Ogata, H., Ayame, H., Nakatsuka, K., Satoh, M., Ukai, M.,…Iio, Y. A benchmarking framework for HVAC optimization via competitive evaluation: insights from the 2nd wccbo. Journal of Building Performance Simulation, 1-14. https://doi.org/10.1080/19401493.2025.2539356